

A

使用区间 DP。设 $f_{l,r,x}$ 表示区间 $[l, r]$ 是否可以满足句法 x 。首先 $f_{i,i,a_i} = 1$ 。

考虑转移，对于区间 $[l, r]$ ，枚举分界点 k ，再枚举规则 $x_i \rightarrow y_i, z_i$ ，如果 f_{l,k,y_i} 和 f_{k+1,r,z_i} 均为 1，那么把 f_{l,r,x_i} 设为 1。

总复杂度 $O(n^3m)$ ，可以通过全部测试数据。

对于部分分：值域小的部分分可以暴力记录区间满足哪些规则，11 ~ 14 因其特殊的规则形式可以直接贪心相邻合并。

B

将 x 的邻居的权值取出来降序排序为 $a_1, a_2, \dots, a_m$ 。

对于 $s = 3, 4, \dots, m$ ：设置初始指针 $p_1 = 1, \dots, p_s = s$ ，以及对应权值 v ，放入堆中，每次取出堆顶，可以选择把最后一个可动指针往后移，或者固定最后一个可动指针并将倒数第二个可动指针后移，同时对应修改权值 v ，放回堆中。注意 $p_1, \dots, p_s$ 不用都记录下来，因为我们每次只移动最后的，再移动次后的，依次移动，所以前缀一定是连续的，只需要记录最最后的尚未移动的指针是哪个即可，以及已经固定的指针中最靠前的是哪个，这样信息量就是 $O(1)$ 的。

再开一个堆统一维护各个 s 当前的最优解，这样可以对每个 x 求出前 k 大了。

再开一个全局的堆汇总每个 x 的答案，即可求出全局的前 k 大。

时间复杂度 $O(n \log n)$ （若认为 n, k 同阶），可以通过全部测试数据。

对于部分分：度数不超过 5 则菊花数量与 n 同阶，可以暴力计算；整个树是个菊花则无需全局堆。

C

要想最小化代价，就是要尽量把 $+$ 边的端点放入同一个集合， $-$ 边的端点放入不同的集合。

不妨先在树上考虑，按照 $k = n, \dots, 1$ 的顺序，最开始每个点自成一个点集，代价就是 $+$ 边的数量。当 k 减小，如果有 $+$ 边的两个端点不在同一个集合，就把他们所在集合合并，这样代价可以减小 1；如果只剩下 $-$ 边，那只能把 $-$ 边两端点所在集合合并，并且代价增加 1。按照我们的贪思路，这样是最优的。

而放在基环树上则略有不同——假设环上有 s 个点，如果我们已经将其中 $s - 1$ 条边的端点合并到同一个集合，那么实际上整个环上的 s 个点均在同一个集合，那么我们不能再将环上剩下的最后一条边的端点合并。并且，还要额外考虑最后一条边产生的代价：

- 如果最后一条边是 $+$ ，那么在合并倒数第二条边时代价额外减小 1；
- 如果最后一条边是 $-$ ，那么在合并倒数第二条边时代价额外增加 1。

又注意到根据贪心， $+$ 一定在 $-$ 之前被选中合并其端点。因此第一种情况出现的唯一可能是整个环都是 $+$ ，这种情况下我们应当优先把环上的边的端点所在集合合并。否则环上存在 $-$ ，一定对应第二种情况，这时：

- 如果环上仅有一条 $-$ 边，那么倒数第二条边一定是 $+$ ，则合并倒数第二条边时代价变化是 $+1 - 1 = 0$ ，所以应当把倒数第二条边留着，先把挂着的数上的 $+$ 合并完之后再合倒数第二条边，最后把树上的 $-$ 边合并；
- 如果环上有多于一条 $-$ 边，那么优先把环上及树上的 $+$ 边端点合并，然后优先把挂着的树上的 $-$ 边端点合并，最后再把环上的 $-$ 边端点合并。

综上所述按照如下顺序把边的端点合并到一个集合，就得到了各个 k 的最优解：

- 如果环上 $-$ 边数量 $\neq 1$ ，则：
 - i. 环上的 $+$ 边；
 - ii. 树上的 $+$ 边；
 - iii. 树上的 $-$ 边；
 - iv. 环上的 $-$ 边。
- 如果环上 $-$ 边数量 $= 1$ ，则：
 - i. 树上的 $+$ 边；
 - ii. 环上的 $+$ 边；
 - iii. 树上的 $-$ 边。

使用并查集模拟这个过程即可得到各个 k 的答案。但是实际上我们计算贡献不必须用并查集——凡是合并 $+$ 均 -1 ，凡是合并 $-$ 均 $+1$ 。但在合并环上的第 $s-1$ 条边时需要额外注意一下（假设环的大小为 s ），如果整个环全是 $+$ 那么代价额外 -1 ，否则代价额外 $+1$ 。找出环上的所有边并记录有多少 $-$ 这件事可以用 DFS 实现，因此放到普及组也是不超纲的。

D

题目已经对所有 S_i 建出 Trie，则 LCP 长度就是两个点在 Trie 上的 LCA 深度，于是两点之间的边权仅与两点本身以及 LCA 有关。

考虑 w_i 全相等的部分分，此时边权与 LCA 是谁无关，我们只需要把所有 a_i 插入一颗 01-Trie，每次在 Trie 上贪心地查询与 $a_i \text{ xor } w_*$ 异或值最小的即可。

那么一般情况我们可以沿用这个思路，只是两个叶子必须在其 LCA 处才能计算贡献。

应用树上启发式合并，需要计算三类贡献：轻子树对轻子树，轻子树对重子树，重子树对轻子树。（注：当前节点自身算在轻子树里边）

轻子树之间的贡献是好计算的，我们只需要暴力把轻子树内的点权插入 01-Trie，然后暴力遍历一个一个查询即可。

重子树对轻子树，按照树上启发式合并的方法，我们会在遍历完重子树后得到一颗含有重子树内所有点权的 01-Trie，此时暴力遍历轻子树内的点，在这颗 Trie 上查询即可。

轻子树对重子树，我们可以把轻子树内所有点权集体异或上 w_x 然后插入 01-Trie，把这颗 Trie 往下传递给重儿子，其内部的点在计算贡献时额外查询一下这颗 Trie 即可。（当然也要继续往下传递给它的重儿子，回溯时撤销插入即可）。

这样复杂度是 $O(n \log n \log V)$ 的，其中 V 是值域，可以通过全部测试数据。

对于链的部分分，短串一定是长串的前缀，所以 w 仅与短串长度有关，于是只需要维护一颗 01-Trie，从前往后边扫边插入，期间在每个点查询最大值，清空后再从后往前做一遍即可。

bonus: 本题原本是求最小生成树，只需要在最外面写一个 Boruvka，然后套用上述算法即可；然而，那样还涉及到要找到不同颜色的边权最小的点，代码将更复杂，且复杂度过高，故最终仅取其子问题考察选手。
